

Hadoop与Spark

Kafka



Kafka

- ◆ Kafka简介

- ◆ Kafka是一个分布式的消息系统(Messaging System)
- ◆ 它最初由LinkedIn公司开发，之后通过Apache基金会开源
- ◆ Kafka是一款分布式的、分区的、和可复制的提交日志服务软件系统；由于其特有的设计，Kafka具有高性能、和高可扩展性的优良特点



Kafka

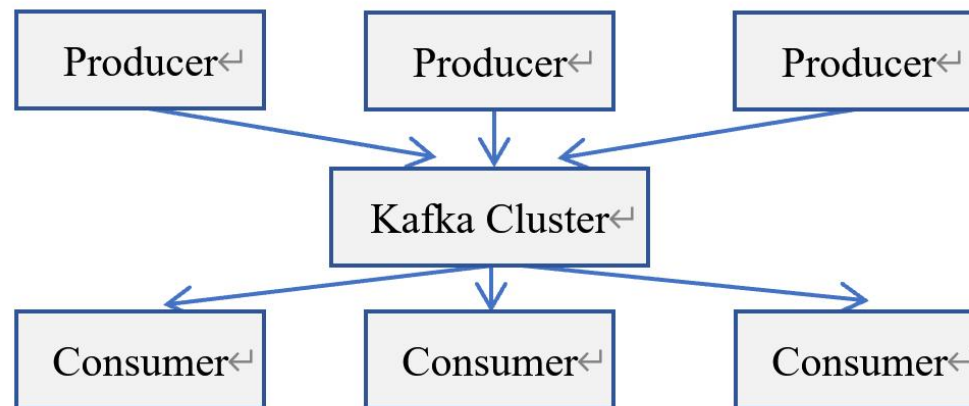
◆ Kafka简介

- ◆ 与传统的消息系统相比，Apache Kafka具有如下的特点：
- ◆ (1) Kafka是一个完全分布式的系统，易于横向扩展，处理极大规模的数据量；
- ◆ (2) Kafka同时为发布和订阅，提供极高的吞吐能力；
- ◆ (3) 它支持多个订阅者，当出现失败状况，可以自动平衡消费者；
- ◆ (4) 它将消息持久化到磁盘，保证消息系统的可靠性
- ◆ Kafka可以用于消息的批量消费应用(比如ETL系统)，以及实时应用程序

Kafka

◆ Kafka的基本概念

- ◆ Kafka消息系统包括如下几个主要组件:
- ◆ (1) Kafka按照类别(称为**主题或者话题Topic**)维护它接收到的消息。话题(Topic)是特定类型的消息流。消息是字节的有效载荷(Payload)
- ◆ (2) 向Kafka发布消息的进程或者程序, 称为**生产者(Producer)**
- ◆ (3) 向Kafka主题进行订阅(Subscribe)、以处理Kafka消息的进程或者程序, 称为**消费者(Consumer)**; 消费者可以订阅一个或多个话题, 从而消费这些已发布的消息
- ◆ (4) 已发布的消息保存在一组服务器中, 它们被称为**代理(Broker)**, 它们共同构成**Kafka集群(Cluster)**



Kafka 的生产者、消费者和代理环境

pause

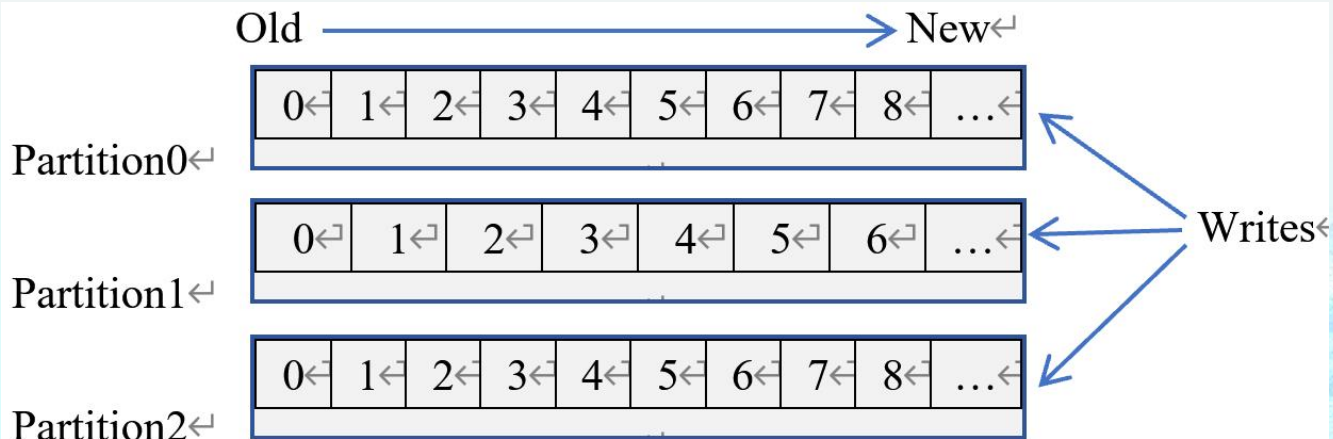


Kafka

◆ Kafka简介

◆ 话题和分区

- ◆ 话题是消息的分类机制，消息生产者要向 Kafka发布消息，必须指定发布到哪个话题(Topic Name)
- ◆ 对于每个话题，Kafka集群维护一个分区的日志文件
- ◆ 话题被划分成一系列分区，每个分区是有序的、不可更改的消息序列、可在末尾不断追加提交日志(Commit Log)
 - ◆ 分区里的每个消息分配了一个顺序号(Sequential ID Number)，唯一地标识分区里的每个消息，这个顺序号也称为偏移量(Offset)



话题与分区

py right π

Kafka

- ◆ Kafka简介
 - ◆ 话题和分区
 - ◆ 在Kafka的消息服务里，使用分区机制，其目的有几个方面
 - ◆ 首先，分区机制允许Kafka处理超过一台服务器容量的日志规模
 - ◆ 每个独立的分区，可以由一个服务器来进行处理，一个话题有很多分区，这些分区分别由不同的服务器来进行处理，那么就使得Kafka在设计上可以处理任意规模的数据量。
 - ◆ 其次，分区是并行处理的基本单元，如果没有分区，就无从进行并行处理

Kafka

- ◆ Kafka简介

- ◆ 偏移量

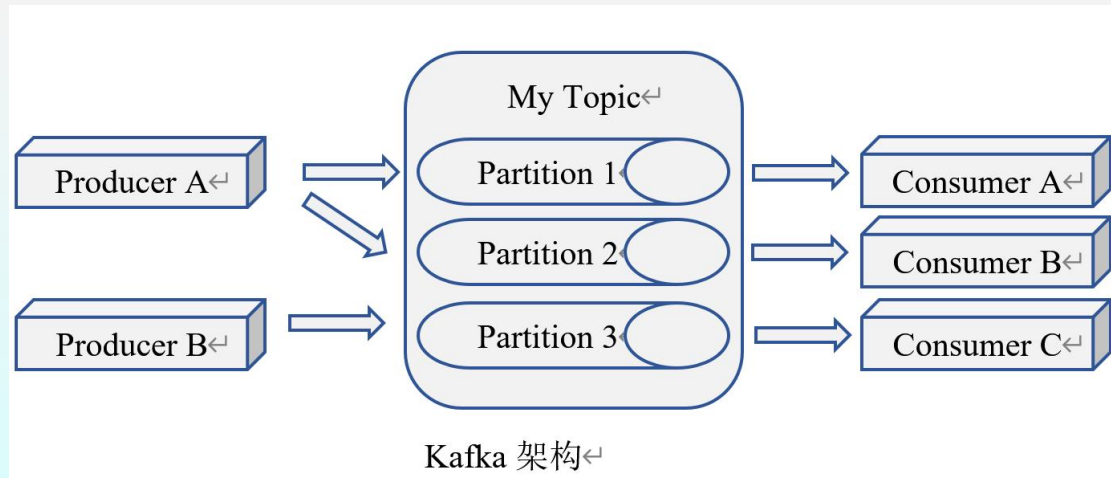
- ◆ 对分区的信息队列的处理，依赖于一个偏移量(Offset)
 - ◆ 偏移量由消费者控制，它指出了每个消费者目前处理到的某个分区的信息队列的位置
 - ◆ 一般来讲，当消费者不断地消费消息以后，偏移量不断地向前推进；但是由于偏移量是由消费者控制的，所以消费者可以以任何顺序来消费消息，也可以把偏移量设定回一个老的偏移量，以便消费已经消费过的消息
 - ◆ 用户甚至可以设计这样的消费者，它仅仅偷偷查看话题的每个分区的末尾若干消息，但是并未改变这些消息，其它的消费者仍然可以正常消费这些消息

Kafka

◆ Kafka简介

◆ Kafka集群

- ◆ 一个Kafka集群通常包括多个代理(Broker)
- ◆ 为了均衡负载，将话题分成多个分区
- ◆ 每个代理存储一个或多个分区
- ◆ 多个生产者和消费者能够同时生产和获取消息



Kafka

◆ Kafka简介

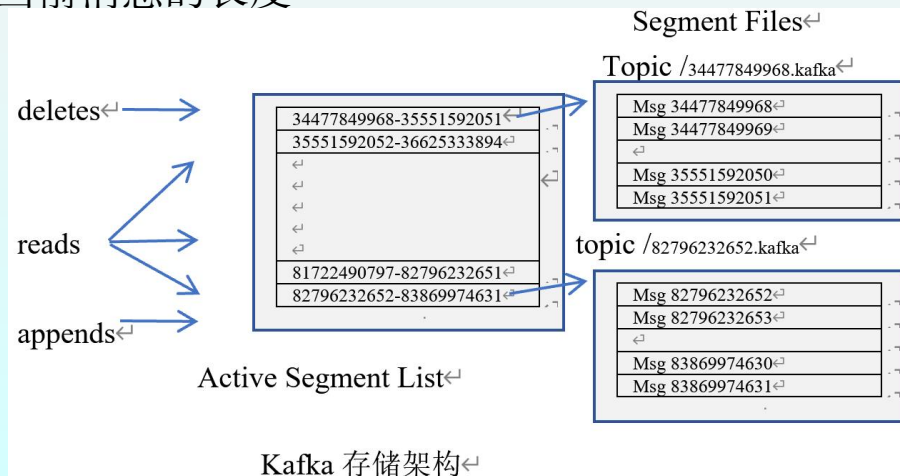
- ◆ 数据分布与存储
- ◆ Kafka把消息分区，分布到Kafka集群的各个服务器上，每个服务器负责这些分区的数据的管理和消息的请求
- ◆ 每个分区的数据，还被复制到一定数量的服务器上(可以配置服务器的数量)，以支持容错
- ◆ 对于每个分区，其中一台服务器作为领导者(Leader)，若干台服务器作为追随者(Follower)
- ◆ 领导者负责分区的读、写请求，追随者以被动地方式(Passively)对领导者的数据进行复制
- ◆ 如果领导者发生失败状况，追随者之一将自动地成为新的领导者
- ◆ 为了保证集群内的负载均衡，每个服务器都担任了一部分分区的领导者、以及一部分分区的追随者的角色

Kafka

◆ Kafka简介

◆ 数据分布与存储

- ◆ Kafka的存储布局非常简单，话题的每个分区对应一个逻辑日志
- ◆ 物理上，一个日志为相同大小的一组分段文件。每次生产者发布消息到一个分区，代理(Broker)就将消息追加到最后一个分段文件中
- ◆ 当发布的消息数量达到设定值或者经过一定的时间后，分段文件真正写入磁盘中。写入完成后，消息公开给消费者。
- ◆ 与传统的消息系统不同，Kafka系统中存储的消息没有明确的消息ID。消息通过日志中的逻辑偏移量来公开
- ◆ 消息ID是增量的，但不连续。要计算下一消息的ID，可以在其逻辑偏移的基础上加上当前消息的长度



Kafka

- ◆ Kafka简介

- ◆ Kafka代理Broker

- ◆ 与其它消息系统不同，Kafka代理是**无状态的**。这意味着消费者必须维护已消费的状态信息。这些信息由消费者自己维护，代理完全不管
 - ◆ 首先，从代理删除消息变得困难，因为代理并不知道消费者是否已经使用了该消息。Kafka创新性地解决了这个问题，它将一个简单的基于时间的SLA(Service Level Agreement)应用于保留策略。当消息在代理中**超过一定时间后，将会被自动删除**
 - ◆ 其次，这种设计有一个很大的好处，消费者可以倒回到老的偏移量再次消费数据
 - ◆ 虽然违反了队列的常见约定，但是在很多实际应用中，表现出这种消费者的特征

Kafka

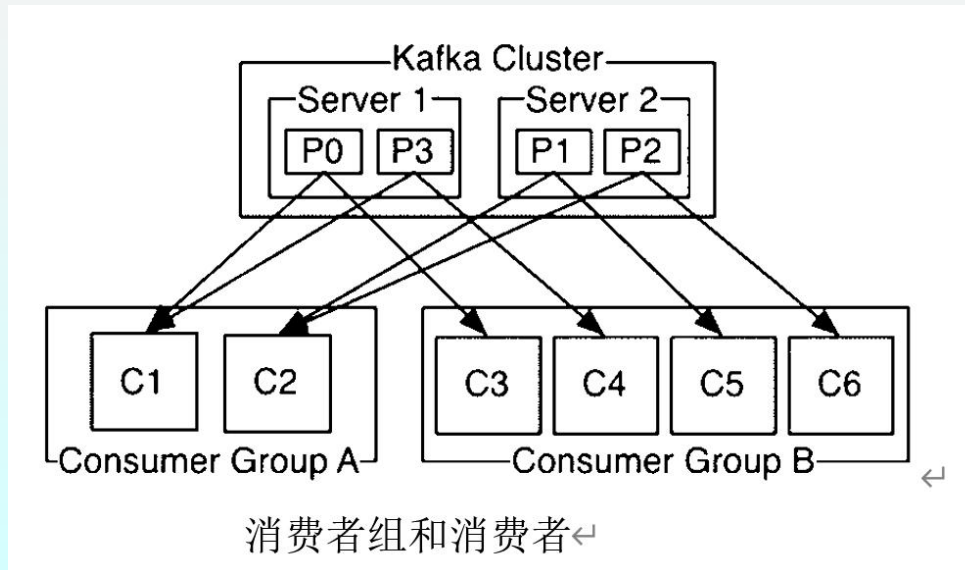
- ◆ Kafka简介
 - ◆ **Kafka生产者**
 - ◆ 生产者向话题发布数据(一般是日志数据)
 - ◆ 由于消息系统的每个话题是分区的, **生产者必须指定把消息分配到哪个分区**
 - ◆ 可以采用Round-Robin的方式, 以便均衡负载
 - ◆ 也可以根据某些应用的语义要求, 设计专用的分区函数(Partition Function), 进行消息的分区

Kafka

◆ Kafka简介

◆ Kafka消费者

- ◆ 下图展示了由两个服务器构成的Kafka集群，管理某个话题的四个分区(P0-P3)，配置了两个消费者组(Group A, Group B)。消费者组A有两个消费者实例，消费者组B有四个消费者实例
- ◆ 从各个分区来看，每个分区的信息，被广播给所有的消费者组，比如分区P0，被广播给消费者组A和消费者组B，分区P1被广播给消费者组A和消费者组B等
- ◆ 另外，在每个消费者组内，发布到组内的消息被各个消费者分担处理了。比如发送到消费者组A的P0和P1分区的信息，被消费者C1处理，而发送到该组的P1和P2分区的信息，则被消费者C2进行处理



Kafka

◆ Kafka简介

◆ 消息的有序性

- ◆ Kafka在存储消息的时候，提供了和传统消息系统一样的顺序保证
 - ◆ 传统的消息队列在服务器上维护消息的顺序，即便有多个消费者从队列中消费这些消息，这些消息以它们存储的顺序，向外递送(Hand Out)
 - ◆ 虽然服务器向外递送消息是按照顺序的，但是消息是异步地递送给各个消费者的，它们到达消费者的顺序有可能被打乱了。也就是说，利用多个消费者进行消息的并行处理的时候，消息的顺序丢失了
 - ◆ 消息系统也可以仅仅使用一个消费进程(独占消费者，Exclusive Consumer)从队列中消费消息，以保证消息被消费的顺序，但是这种方法全丢了并行性
- 需要注意的是，Kafka仅仅提供了**分区内消息的有序性**，并不能保证一个话题的不同分区之间的消息的有序性
 - 对于大量的实际应用来讲，在分区内保证消息的有序性，和数据的分区处理已经足够了
 - 但是，如果用户需要在所有消息上实现顺序处理，只能在一个话题里有一个分区，而且只能由一个消费者进行处理

pause



Kafka

- ◆ Kafka简介
 - ◆ 应用场景
 - ◆ Kafka以高度的扩展能力和高性能，获得了极大的关注，并且在大量的实际系统中得到应用。
- 消息传递(Messaging)
 - ◆ 在消息处理领域，Kafka可以作为传统消息队列的替代品，比如ActiveMQ和RabbitMQ。使用消息代理(Message Broker)，可以把紧密耦合的系统设计给解耦，可以对未及时处理的消息进行缓存
 - ◆ 相对于大部分传统消息系统，Kafka具有更高的吞吐能力，并且提供了分区、复制、以及容错的支持，使得它成为大型消息处理系统的优先选择
 - ◆ 在某些应用场合，并不需要很高的吞吐能力，但是要求消息处理的延迟要尽量小，并且需要系统提供消息的可保证，Kafka的高性能以及持久化能力，正好提供这样的支持

Kafka

- ◆ Kafka简介
 - ◆ 应用场景
- **网站活动跟踪(Website Activity Tracking)**
 - ◆ 网站上的用户活动，包括查看页面(Page Views)、搜索(Search)、以及其它动作，可以发布到一些话题上，每类用户活动对应一个话题
 - ◆ 这些消息，可以被一些消费者程序进行订阅和处理，包括一些实时处理程序比如实时监控，也可以装载到Hadoop或者数据仓库系统，以便进行离线式的批处理以及报表生成
 - ◆ 在用户活动跟踪过程中，为了记录大量的用户活动，系统必须能够处理快速产生的消息，每个消息对应用户的一次页面点击、或者一次搜索操作
 - ◆ Kafka的高性能为大量消息的及时处理，提供了保证

Kafka

- ◆ Kafka简介
 - ◆ 应用场景
- **度量(Metric)**
 - ◆ Kafka经常应用在业务运营数据的监控与汇总中
 - ◆ 在这类应用中，需要把各个应用程序的数据，集中到一起，然后在它上边进行汇总统计
- **日志聚集**
 - ◆ 日志聚集应用，一般从各个服务器收集日志文件，然后保存到一个集中的存储位置，比如一个文件服务器，或者HDFS，然后进行后续的处理
 - ◆ Kafka通过抽象，把日志文件里面的日志或者事件，看作是一个消息流(Stream of Message)
 - ◆ Kafka的设计支持低延迟地处理多数据源的数据
 - ◆ 相对于Scribe和Flume等日志处理系统，Kafka提供足够好的性能，即它提供更低的处理延迟，并且保证消息的持久性

Kafka

- ◆ Kafka简介
 - ◆ 应用场景
- **流数据处理(Stream Processing)**
 - ◆ 和Storm和Samza等系统一样, Kafka同样可以使用于流数据处理
 - ◆ 在某些应用中, 用户对数据的处理是分阶段进行的, 首先从各个数据源进行数据聚集, 然后对数据进行标注和语义丰富(Enrich), 进行必要数据转换(Transform), 以便后续的消费和使用
 - ◆ 比如, 在文章推荐应用中, 整个处理流程大概可以分为如下几个阶段
 - ◆ 首先从RSS数据源爬取(Crawl)各类文章, 发布到“Article”主题
 - ◆ 然后, 对这些文章进行规范化(Normalize)、去重(De-Duplicate)等处理, 发布到“Clean Article”主题
 - ◆ 最后, 可以根据用户的喜好, 把文章推荐给用户
 - ◆ 这些处理阶段, 构成了一个实时数据处理流程, 把各个主题链接起来

Kafka

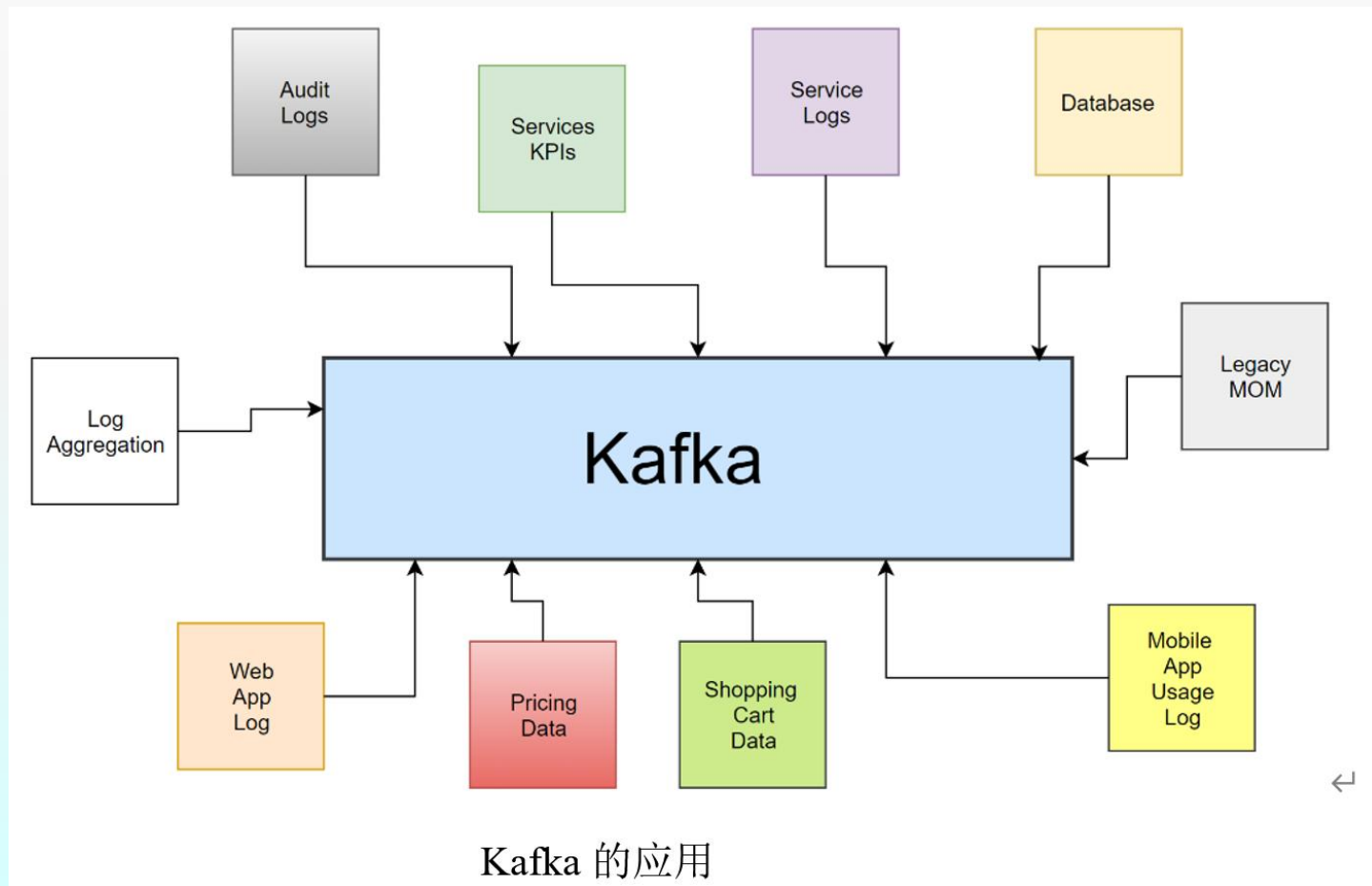
- ◆ Kafka简介
 - ◆ 应用场景
- **事件溯源Event Sourcing**
 - ◆ 事件溯源，是把应用程序状态的变化，以具有时间顺序的(每个记录有时间戳)一系列消息，记录下来
 - ◆ 换句话说，即保存一个对象所经历的每个事件，所有由该对象产生的事件都按照时间先后顺序有序地存放在数据库中
 - ◆ Kafka支持大量日志数据的存储，使得它成为事件溯源系统的后端数据存储的可靠选择

Kafka

- ◆ Kafka简介
 - ◆ 应用场景
- **提交日志Commit Log**
 - ◆ Kafka可以作为分布式系统提交日志的外部存储服务
 - ◆ 在分布式系统中，这些日志帮助节点之间进行数据复制，并且在节点失败后，利用日志作为重新同步的机制，帮助节点把数据恢复回来
 - ◆ Kafka支持日志压缩(Log Compaction)功能，支持上述应用场景；在这个应用场景中，Kafka提供了类似于Apache BookKeeper的功能
- ◆ 简而言之，Kafka的应用场景很广泛，包括暂时存储各种类型的数据包括网站活动跟踪(website activity tracking)、日志集中和聚集(log aggregation)、传感器数据收集和监控(metrics collection and monitoring)，以及把数据转移到Hadoop和Spark平台等
- ◆ Kafka可以保存提交日志(commit log for in-memory computing)，保证内存数据处理系统的可恢复性和可靠性

Kafka

- ◆ Kafka简介
 - ◆ 应用场景



Kafka

- ◆ Kafka简介

- ◆ 小结

- ◆ Kafka是一款支持大量数据处理的消息系统。它提供低延迟、高吞吐量的消息处理，并且保证消息的持久性
 - ◆ Kafka基于拉(Pull)的消费模型，让消费者以自己的速度处理消息
 - ◆ 如果处理消息时出现了异常，消费者始终可以选择再消费该消息

pause

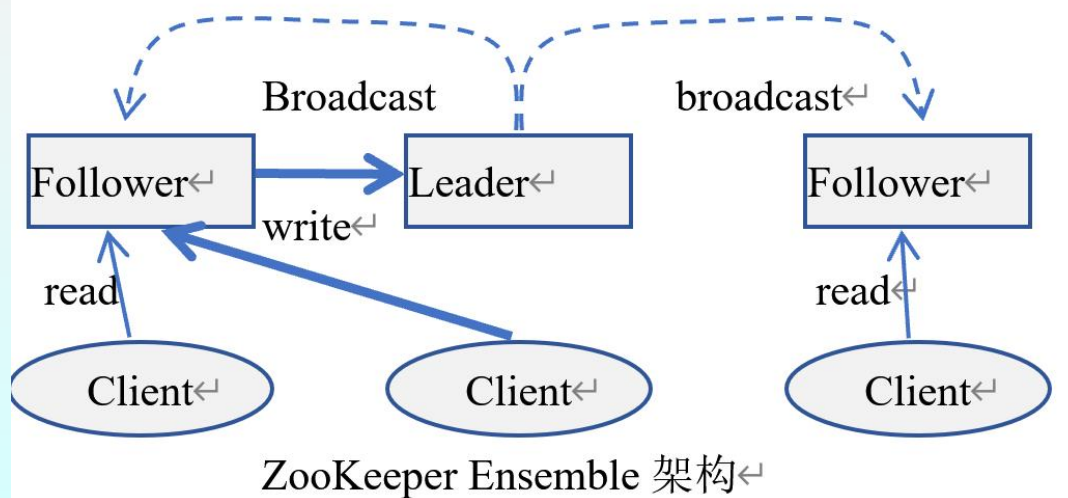


Kafka

- ◆ Kafka简介
 - ◆ Kafka与zookeeper
 - ◆ Zookeeper是一款协调服务软件
 - ◆ 我们可以使用Zookeeper构建可靠的、分布式的数据结构
 - ◆ 提供群组成员维护、领导人选举、工作流协同、分布式系统同步、命名、和配置信息维护等服务，以及广义的分布式数据结构如锁、队列、屏障(Barrier)和锁存器(Latch)
 - ◆ 许多知名的项目依赖于 Zookeeper，包括HBase

Kafka

- ◆ Kafka简介
 - ◆ Kafka与zookeeper
 - ◆ 下图展示了典型的Zookeeper Ensemble，一台服务器作为Leader，其它作为Follower
 - ◆ 当Ensemble启动时，先选出Leader，然后所有Follower复制Leader的状态；**所有写请求都通过Leader路由**，变更会广播给所有Follower
 - ◆ 变更广播被称为原子广播

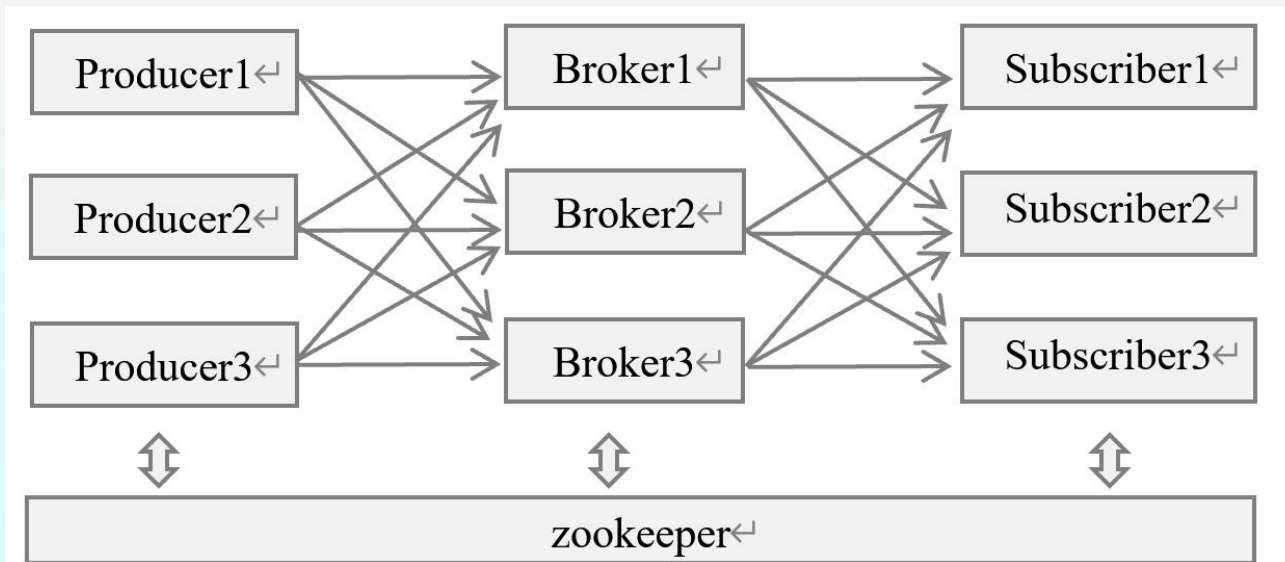


Kafka

◆ Kafka简介

◆ Kafka与zookeeper

- ◆ 每个Kafka代理都通过Zookeeper与其它 Kafka代理协同
- ◆ 当Kafka系统中新增了代理或者某个代理故障失效时，Zookeeper服务将通知生产者和消费者
- ◆ 生产者和消费者据此开始与其它代理协同工作

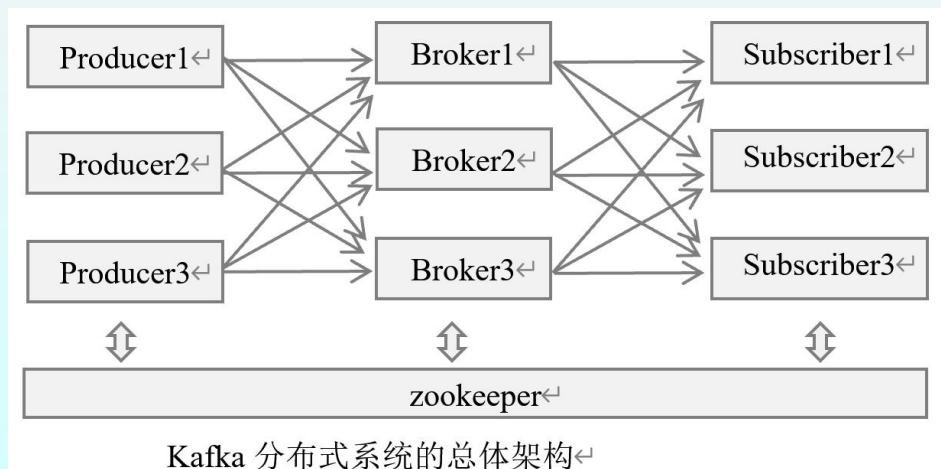


Kafka 分布式系统的总体架构

Kafka

◆ Kafka简介: Zookeeper

- ◆ Zookeeper是一个分布式的协调服务，Kafka集群依赖于Zookeeper来存储和管理Kafka的元数据信息和配置信息，在Kafka集群中，**Zookeeper主要负责以下任务**
- ◆ (1) 存储和管理Kafka集群的元数据信息和配置信息，包括broker的IP地址、端口号、主题分区的分配方案等
- ◆ (2) 管理Kafka broker节点的状态信息，包括broker的上/下线状态、主题分区信息、副本信息等
- ◆ (3) 通过监控Kafka broker节点的状态信息，帮助Kafka集群实现自动故障转移和负载均衡等功能
- ◆ (4) Leader选举：每个partition都有一个Leader
- ◆ (5) 消费者offset信息、消费者组维护/消费者组变化时重新进行rebalance



Kafka

- ◆ Kafka简介

- ◆ 流数据处理

- ◆ Kafka Streams支持对数据流进行实时处理
 - ◆ Kafka Stream API可以对多个数据流进行聚集、连接等操作，支持有状态的计算
 - ◆ 它提供了Stream Processor，把某个topic的数据以持续的数据流的形式看待和处理，主要的处理包括转换、聚集等
 - ◆ Stream Processor产生多个输出流(Output Streams)
 - ◆ 比如，在视频播放应用中，可以监控用户的一系列事件，包括开始观看视频、暂停等，在此基础上进行分析，了解用户的偏好
 - ◆ 基于某个用户的行为、或者很多用户的行为，了解热门的视频节目和片段，改善视频推荐

pause

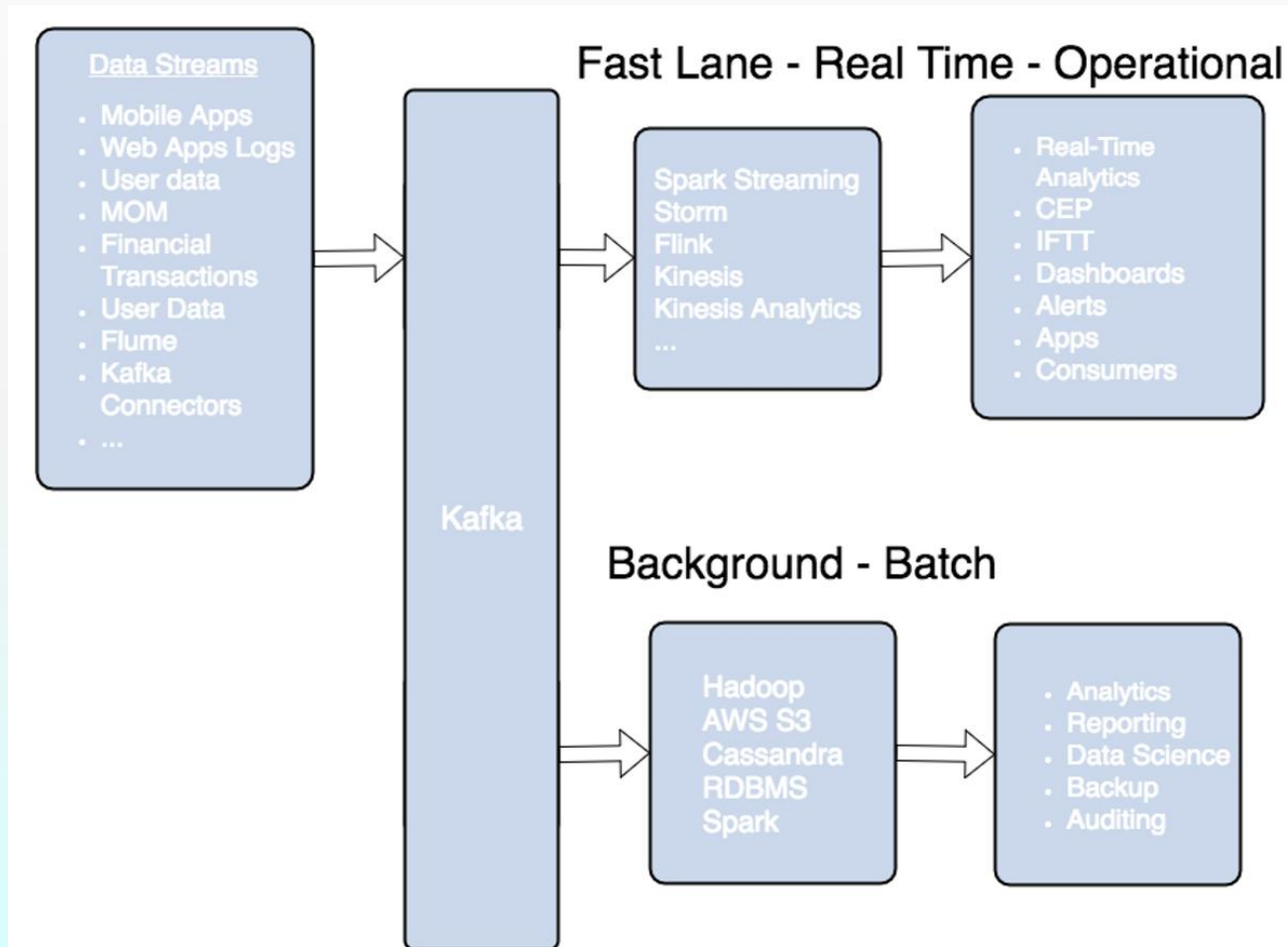


Kafka

- ◆ Kafka在系统中的位置
 - ◆ 一般来讲，Kafka用于暂存快速到达的数据，保证数据不丢
 - ◆ 然后通过其它组件，把数据迁移到其它系统比如Hadoop和Spark
 - ◆ Kafka处于中间层，它解开快速数据流和后续的流式处理和批量处理系统的耦合
 - ◆ Kafka通过Kafka Streaming 支持流式数据的处理，但是Kafka核心组件是不支持进行流式数据处理的
 - ◆ 我们可以把Kafka暂时存储的数据，拉取到流式数据处理系统里进行实时分析，比如Storm、Flink、Spark Streaming等
 - ◆ Kafka暂存的数据，也可以拉取到Hadoop里，以便进行批量处理。当然也可以把Kafka的数据转移到RDBMS、Cassandra、Spark等系统
 - ◆ 在这些系统里进行数据的统计汇总和报表生成，以及各种深入分析

Kafka

◆ Kafka在系统中的位置



Kafka 在系统中的位置

pause



Kafka

- ◆ **Kafka**安装、配置与运行
- ◆ 下载 kafka_2.10-0.10.2.1
- ◆ 解压和授权

```
cd /opt/linuxsir  
tar -xzf kafka_2.10-0.10.2.1.tgz  
  
cd kafka_2.10-0.10.2.1  
chmod -R a+rwX *  
chown -R nobody:nobody *
```

- Kafka需要用到ZooKeeper，在这里我们使用内置的Zookeeper
- 在生产环境下，建议安装独立的Zookeeper

Kafka

- ◆ **Kafka**安装、配置与运行
- ◆ 查看config/zookeeper.properties配置文件，确认其内容如下

```
# the directory where the snapshot is stored.
```

```
dataDir=/tmp/zookeeper
```

```
# the port at which the clients will connect
```

```
clientPort=12182
```

```
# disable the per-ip limit on the number of connections since this is a non-  
production config
```

```
maxClientCnxns=0
```

注意Zookeeper的端口改成12182，因为
HBase Zookeeper使用了12181

Kafka

- ◆ **Kafka**安装、配置与运行
- ◆ 修改config/server.properties配置文件，具体如下

```
# root directory for all kafka znodes.  
zookeeper.connect=192.168.31.129:12182  
  
# Timeout in ms for connecting to zookeeper  
zookeeper.connection.timeout.ms=6000
```

Kafka

- ◆ **Kafka**安装、配置与运行
- ◆ 启动后台服务器，包括启动Zookeeper和broker 0

```
cd /opt/linuxsir/kafka_2.10-0.10.2.1  
bin/zookeeper-server-start.sh config/zookeeper.properties &
```

```
\\ 等待zookeeper启动  
netstat -ntlp|grep 12182
```

```
bin/kafka-server-start.sh config/server.properties &
```

```
\\ 等待kafka broker 0启动  
netstat -ntlp|grep 9092  
\\ 注意Kafka Broker在192.168.31.129启动之后，在9092端口监听
```

Kafka

- ◆ **Kafka**安装、配置与运行
- ◆ 创建Kafka主题

```
bin/kafka-topics.sh --create --zookeeper 192.168.31.129:12182 --replication-factor 1 --partitions 1 --topic test1
```

```
bin/kafka-topics.sh --list --zookeeper 192.168.31.129:12182
```

\\ 可以用下面的命令删除主题

```
bin/kafka-topics.sh --delete --zookeeper 192.168.31.129:12182 --topic test1
```

Kafka

- ◆ **Kafka**安装、配置与运行
- ◆ 启动一个终端，运行如下命令(Producer)

```
cd /opt/linuxsir  
cd kafka_2.10-0.10.2.1  
bin/kafka-console-producer.sh --broker-list 192.168.31.129:9092 --topic test1
```

在生产者端，不断输入和回车
敲Ctrl+C结束输入

Kafka

- ◆ **Kafka**安装、配置与运行
- ◆ 启动另外一个终端，运行如下命令(Consumer)

```
cd /opt/linuxsir  
cd kafka_2.10-0.10.2.1  
bin/kafka-console-consumer.sh --zookeeper 192.168.31.129:12182 --topic test1 -  
-from-beginning  
\\ 或bin/kafka-console-consumer.sh --bootstrap-server 192.168.31.129:9092 --  
topic test1 --from-beginning
```

随着用户不断在Producer终端窗口输入一行行内容和回车，Consumer终端窗口不断显示刚刚输入的内容。可以在Producer终端窗口敲Ctrl+C结束输入

Kafka

- ◆ **Kafka**安装、配置与运行
- ◆ 停止broker和zookeeper

```
cd /opt/linuxsir/kafka_2.10-0.10.2.1  
bin/zookeeper-server-stop.sh config/zookeeper.properties &  
bin/kafka-server-stop.sh config/server.properties &
```

只有如下命令不能停止服务器的时候，才使用杀死进程的方法

```
netstat -ntlp| grep 12182  
kill -9 60465 \\ 60465为zookeeper的进程号
```

```
netstat -ntlp| grep 9092  
kill -9 60714 \\ 60714为broker的进程号
```

pause



Kafka

- ◆ Kafka安装、配置与运行
- ◆ 多broker配置
- ◆ 从config/server.properties拷贝config/server-1.properties和config/server-2.properties

```
cd /opt/linuxsir/kafka_2.10-0.10.2.1  
cp config/server.properties config/server-1.properties  
cp config/server.properties config/server-2.properties
```

Kafka

- ◆ **Kafka安装、配置与运行**
- ◆ 多broker配置
 - ◆ 编辑config/server-1.properties和config/server-2.properties。
- ◆ config/server-1.properties需要改动的内容如下

```
broker.id=1  
listeners = PLAINTEXT://192.168.31.129:9093  
log.dirs=/tmp/kafka-logs1
```

- ◆ config/server-2.properties需要改动的内容如下

```
broker.id=2  
listeners = PLAINTEXT://192.168.31.129:9094  
log.dirs=/tmp/kafka-logs2
```

Kafka

- ◆ Kafka安装、配置与运行
- ◆ 多broker配置
 - ◆ 启动后台服务器，包括启动Zookeeper和broker 0

```
cd /opt/linuxsir/kafka_2.10-0.10.2.1  
bin/zookeeper-server-start.sh config/zookeeper.properties &
```

```
\\ 等待zookeeper启动  
netstat -ntlp|grep 12182
```

```
cd /opt/linuxsir/kafka_2.10-0.10.2.1  
bin/kafka-server-start.sh config/server.properties &
```

```
\\ 等待kafka broker 0启动  
netstat -ntlp|grep 9092
```

```
\\ 注意Kafka Broker在192.168.31.129启动之后，在9092端口监听
```

Kafka

- ◆ Kafka安装、配置与运行
- ◆ 多broker配置
 - ◆ 启动broker1和broker2

```
cd /opt/linuxsir/kafka_2.10-0.10.2.1  
bin/kafka-server-start.sh config/server-1.properties &  
bin/kafka-server-start.sh config/server-2.properties &
```

```
\\ 等待kafka broker1和broker2启动  
netstat -ntlp|grep 9093  
netstat -ntlp|grep 9094
```

Kafka

- ◆ Kafka安装、配置与运行
- ◆ 多broker配置
 - ◆ 创建一个Topic，复制因子为3

```
bin/kafka-topics.sh --create --zookeeper 192.168.31.129:12182 --replication-factor 3 --partitions 1 --topic my-replicated-topic
```

Kafka

- ◆ Kafka安装、配置与运行
- ◆ 多broker配置
 - ◆ 通过Zookeeper了解每个Broker工作状态

```
bin/kafka-topics.sh --describe --zookeeper 192.168.31.129:12182 --topic my-replicated-topic
```

显示结果如下，表示Leader是broker0。

```
Topic:my-replicated-topic    PartitionCount:1    ReplicationFactor:3    Configs:
    Topic: my-replicated-topic    Partition: 0    Leader: 0    Replicas: 0,2,1    Isr:
0,2,1
```

Kafka

- ◆ Kafka安装、配置与运行
- ◆ 多broker配置
 - ◆ 启动一个终端，运行如下命令(Producer)

```
cd /opt/linuxsir/kafka_2.10-0.10.2.1
bin/kafka-console-producer.sh --broker-list 192.168.31.129:9092 --topic my-
replicated-topic
输入如下内容
my test message 1
my test message 2
^C
```

Kafka

- ◆ Kafka安装、配置与运行
- ◆ 多broker配置
 - ◆ 启动另外一个终端，运行如下命令(Consumer)

```
cd /opt/linuxsir/kafka_2.10-0.10.2.1  
bin/kafka-console-consumer.sh --bootstrap-server 192.168.31.129:9092 --from-  
beginning --topic my-replicated-topic  
\\或者  
bin/kafka-console-consumer.sh --zookeeper 192.168.31.129:12182 --from-  
beginning --topic my-replicated-topic
```

随着用户不断在Producer终端窗口输入一行行内容和回车，Consumer终端窗口不断显示刚刚输入的内容。可以在Producer终端窗口敲Ctrl+C结束输入

pause



Kafka

- ◆ Kafka安装、配置与运行
- ◆ 多broker配置
 - ◆ 测试容错性
 - ◆ 接着上一节的实验，我们可以测试系统的容错性。
- ◆ Broker 0是leader，我们试试把它kill掉。查看broker0的进程号，然后把它kill掉

```
ps aux | grep server.properties  
netstat -nltp|grep 9092  
kill -9 65072
```

Kafka

- ◆ Kafka安装、配置与运行
- ◆ 多broker配置
 - ◆ 测试容错性
 - ◆ 再查看my-replicated-topic的状况。

```
cd /opt/linuxsir/kafka_2.10-0.10.2.1  
bin/kafka-topics.sh --describe --zookeeper 192.168.31.129:12182 --topic my-  
replicated-topic
```

结果如下，leader已经变成broker2。

```
Topic:my-replicated-topic      PartitionCount:1  
ReplicationFactor:3  Configs:  
      Topic: my-replicated-topic      Partition: 0      Leader: 2  
Replicas: 0,2,1  Isr: 2,1
```

Kafka

- ◆ Kafka安装、配置与运行
- ◆ 多broker配置
 - ◆ 测试容错性
 - ◆ 从头查看消息，一点问题没有

```
cd /opt/linuxsir/kafka_2.10-0.10.2.1
bin/kafka-console-consumer.sh --zookeeper 192.168.31.129:12182 --from-
beginning --topic my-replicated-topic
\\或者连接到broker1或者broker2看看(broker0被杀掉了)
bin/kafka-console-consumer.sh --bootstrap-server 192.168.31.129.:9093 --from-
beginning --topic my-replicated-topic
```

pause



Kafka

- ◆ Kafka安装、配置与运行
- ◆ 安装问题1: zookeeper日志越来越大

- Zookeeper日志越积越多，占用空间
- Zookeeper运行过程中，不断累积日志。如果不及时清理，占用极大硬盘空间。
- 可以考虑安装独立的zookeeper，版本大于3.4.0.0，可以配置它自动purge日志

pause



Kafka

- ◆ **Java开发**
- ◆ 下面，介绍如何用Java语言编写Kafka Producer和Consumer，以及如何用Eclipse进行调试
- ◆ (1) 新建Maven项目(Kafka Producer)
- ◆ 关于如何安装Maven，给Eclipse安装和配置Maven插件，以及新建Maven项目，请参考教材8.4节

```
package kafka_producer.kafka_producer;  
  
import org.apache.kafka.clients.producer.KafkaProducer;  
import org.apache.kafka.clients.producer.ProducerConfig;  
import org.apache.kafka.clients.producer.ProducerRecord;  
import org.apache.kafka.common.serialization.StringSerializer;  
  
import java.util.Properties;
```

Kafka

- ◆ Java开发
- ◆ (1) 新建Maven项目(Kafka Producer)

```
public class VerySimpleProducer {  
  
    // The topic we are going to write records to  
    private static final String KAFKA_TOPIC_NAME = "very_simple_topic";  
  
    public static void main(String[] args) {  
        // Set producer configuration properties  
        final Properties producerProps = new Properties();  
        producerProps.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, "192.168.31.129:9092");  
        producerProps.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, StringSerializer.class);  
        producerProps.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, StringSerializer.class);  
  
        // Create a new producer  
        final KafkaProducer<String, String> producer = new KafkaProducer<>(producerProps);  
        // Write 10 records into the topic  
        for (int i = 0; i < 10; i++) {  
            final String key = "key-" + i;  
            final String value = "value-" + i;  
            producer.send(new ProducerRecord<>(KAFKA_TOPIC_NAME, key, value));  
        }  
    }  
}
```

Kafka

- ◆ Java开发
- ◆ (1) 新建Maven项目(Kafka Producer)——pom.xml文件配置如下

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>kafka_producer</groupId>
  <artifactId>kafka_producer</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <packaging>jar</packaging>

  <name>kafka_producer</name>
  <url>http://maven.apache.org</url>

  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>
```

Kafka

- ◆ Java开发
- ◆ (1) 新建Maven项目(Kafka Producer)——pom.xml文件配置如下

```
<dependencies>
  <dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>3.8.1</version>
    <scope>test</scope>
  </dependency>

  <dependency>
    <groupId>org.apache.kafka</groupId>
    <artifactId>kafka-clients</artifactId>
    <version>0.10.2.1</version>
  </dependency>

  <dependency>
    <groupId>org.apache.kafka</groupId>
    <artifactId>kafka_2.10</artifactId>
    <version>0.10.2.1</version>
  </dependency>
</dependencies>
</project>
```

Kafka

- ◆ Java开发
- ◆ (2) 新建Maven项目 (Kafka Consumer)
- ◆ Consumer 的Java代码如下

```
package kafka_consumer.kafka_consumer;

import org.apache.kafka.clients.consumer.ConsumerConfig;
import org.apache.kafka.clients.consumer.ConsumerRecord;
import org.apache.kafka.clients.consumer.ConsumerRecords;
import org.apache.kafka.clients.consumer.KafkaConsumer;
import org.apache.kafka.clients.producer.KafkaProducer;
import org.apache.kafka.clients.producer.ProducerConfig;
import org.apache.kafka.clients.producer.ProducerRecord;
import org.apache.kafka.common.serialization.StringDeserializer;
import org.apache.kafka.common.serialization.StringSerializer;

import java.util.Collections;
import java.util.Properties;
```

Kafka

- ◆ Java开发
- ◆ (2) 新建Maven项目(Kafka Consumer)
- ◆ Consumer 的Java代码如下

```
public class VerySimpleConsumer {  
  
    // The topic we are going to read records from  
    private static final String KAFKA_TOPIC_NAME = "very_simple_topic";  
  
    public static void main(String[] args) {  
        // Set consumer configuration properties  
        final Properties consumerProps = new Properties();  
        consumerProps.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, "192.168.31.129:9092");  
        consumerProps.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class);  
        consumerProps.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class);  
        consumerProps.put(ConsumerConfig.GROUP_ID_CONFIG, "very-simple-consumer");  
  
        // Create a new consumer  
        final KafkaConsumer<String, String> consumer = new KafkaConsumer<>(consumerProps);  
        // Subscribe to the topic  
        consumer.subscribe(Collections.singleton(KAFKA_TOPIC_NAME));  
  
        // Continuously read records from the topic  
        while (true) {  
            final ConsumerRecords<String, String> records = consumer.poll(1000);  
            for (ConsumerRecord<String, String> record : records) {  
                System.out.println("Received: " + record);  
            }  
        }  
    }  
}
```

Kafka

- ◆ Java开发
- ◆ (2) 新建Maven项目(Kafka Consumer)
- ◆ pom.xml文件配置如下

和producer项目的pom.xml文件是一样的

Kafka

- ◆ **Java开发**
 - ◆ **(3)启动ZooKeeper和Kafka Broker 0**
- ◆ 使用如下命令，启动ZooKeeper和Kafka Broker 0

```
cd /opt/linuxsir/kafka_2.10-0.10.2.1
bin/zookeeper-server-start.sh config/zookeeper.properties &

\\ 等待zookeeper启动
netstat -ntlp|grep 12182

bin/kafka-server-start.sh config/server.properties &

\\ 等待kafka broker 0启动
netstat -ntlp|grep 9092
\\ 注意Kafka Broker在192.168.31.129启动之后，在9092端口监听
```

Kafka

- ◆ **Java开发**
 - ◆ **(4) 创建一个Kafka Topic**
- ◆ 创建Kafka Topic, 命名为very_simple_topic

```
cd /opt/linuxsir/kafka_2.10-0.10.2.1  
bin/kafka-topics.sh --create --zookeeper 192.168.31.129:12182 --replication-  
factor 1 --partitions 1 --topic very_simple_topic  
bin/kafka-topics.sh --list --zookeeper 192.168.31.129:12182
```

Kafka

- ◆ Java开发
 - ◆ (5) Run或者Debug Consumer
 - ◆ 在Eclipse里，为Consumer项目新建Run Configuration，然后运行程序，或者新建Debug Configuration，然后调试程序
 - ◆ 新建Run Configuration的命令和菜单序列为，对着项目→按右键→Run As→Run Configurations→双击Java Applications新建一个Run Configuration→Main Tab选择Project→Main Tab选择Main Class”，然后选择Run按钮即可
 - ◆ 新建Run Configuration的命令和菜单序列类似

Kafka

◆ Java开发

◆ (6) Run或者Debug Producer

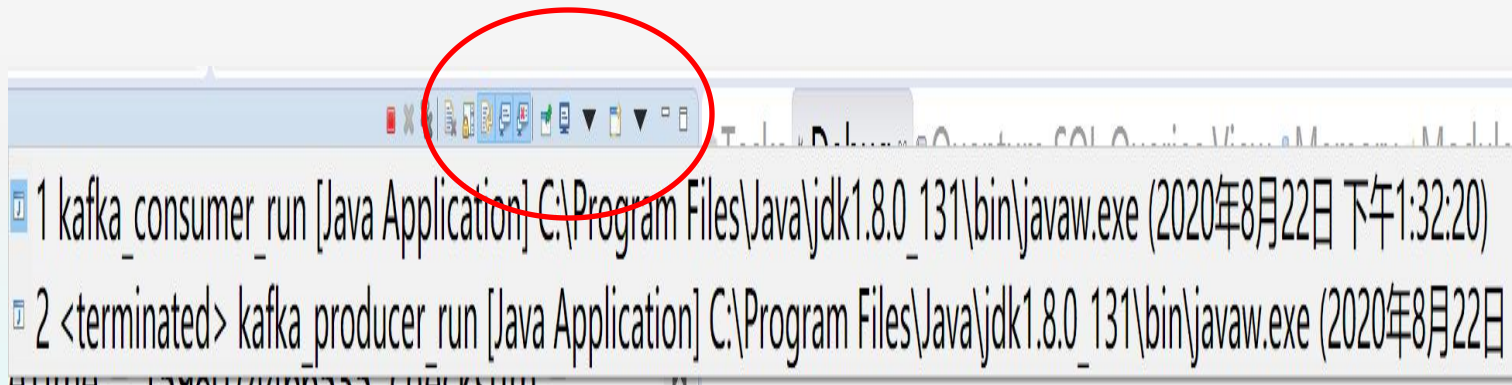
◆ 在Eclipse里，为Consumer项目新建Run Configuration，然后运行程序，或者新建Debug Configuration，然后调试程序

◆ 在Producer端生成10个Key Value，可以在Consumer端看到相应的输出

```
Received: ConsumerRecord(topic = very_simple_topic, partition = 1, offset = 0, CreateTime = 1513475276740, checksum = 1161559315, serialized key size = 5, serialized value size = 7, key = key-0, value = value-0)
Received: ConsumerRecord(topic = very_simple_topic, partition = 1, offset = 1, CreateTime = 1513475276747, checksum = 3822194193, serialized key size = 5, serialized value size = 7, key = key-7, value = value-7)
Received: ConsumerRecord(topic = very_simple_topic, partition = 1, offset = 2, CreateTime = 1513475276747, checksum = 33131761, serialized key size = 5, serialized value size = 7, key = key-8, value = value-8)
Received: ConsumerRecord(topic = very_simple_topic, partition = 2, offset = 0, CreateTime = 1513475276747, checksum = 186355086, serialized key size = 5, serialized value size = 7, key = key-2, value = value-2)
Received: ConsumerRecord(topic = very_simple_topic, partition = 2, offset = 1, CreateTime = 1513475276747, checksum = 3887722871, serialized key size = 5, serialized value size = 7, key = key-3, value = value-3)
Received: ConsumerRecord(topic = very_simple_topic, partition = 2, offset = 2, CreateTime = 1513475276747, checksum = 3790074274, serialized key size = 5, serialized value size = 7, key = key-5, value = value-5)
Received: ConsumerRecord(topic = very_simple_topic, partition = 2, offset = 3, CreateTime = 1513475276747, checksum = 259027688, serialized key size = 5, serialized value size = 7, key = key-6, value = value-6)
Received: ConsumerRecord(topic = very_simple_topic, partition = 0, offset = 0, CreateTime = 1513475276746, checksum = 2052492634, serialized key size = 5, serialized value size = 7, key = key-1, value = value-1)
Received: ConsumerRecord(topic = very_simple_topic, partition = 0, offset = 1, CreateTime = 1513475276747, checksum = 222661979, serialized key size = 5, serialized value size = 7, key = key-4, value = value-4)
Received: ConsumerRecord(topic = very_simple_topic, partition = 0, offset = 2, CreateTime = 1513475276747, checksum = 3982225416, serialized key size = 5, serialized value size = 7, key = key-9, value = value-9)
```

Kafka

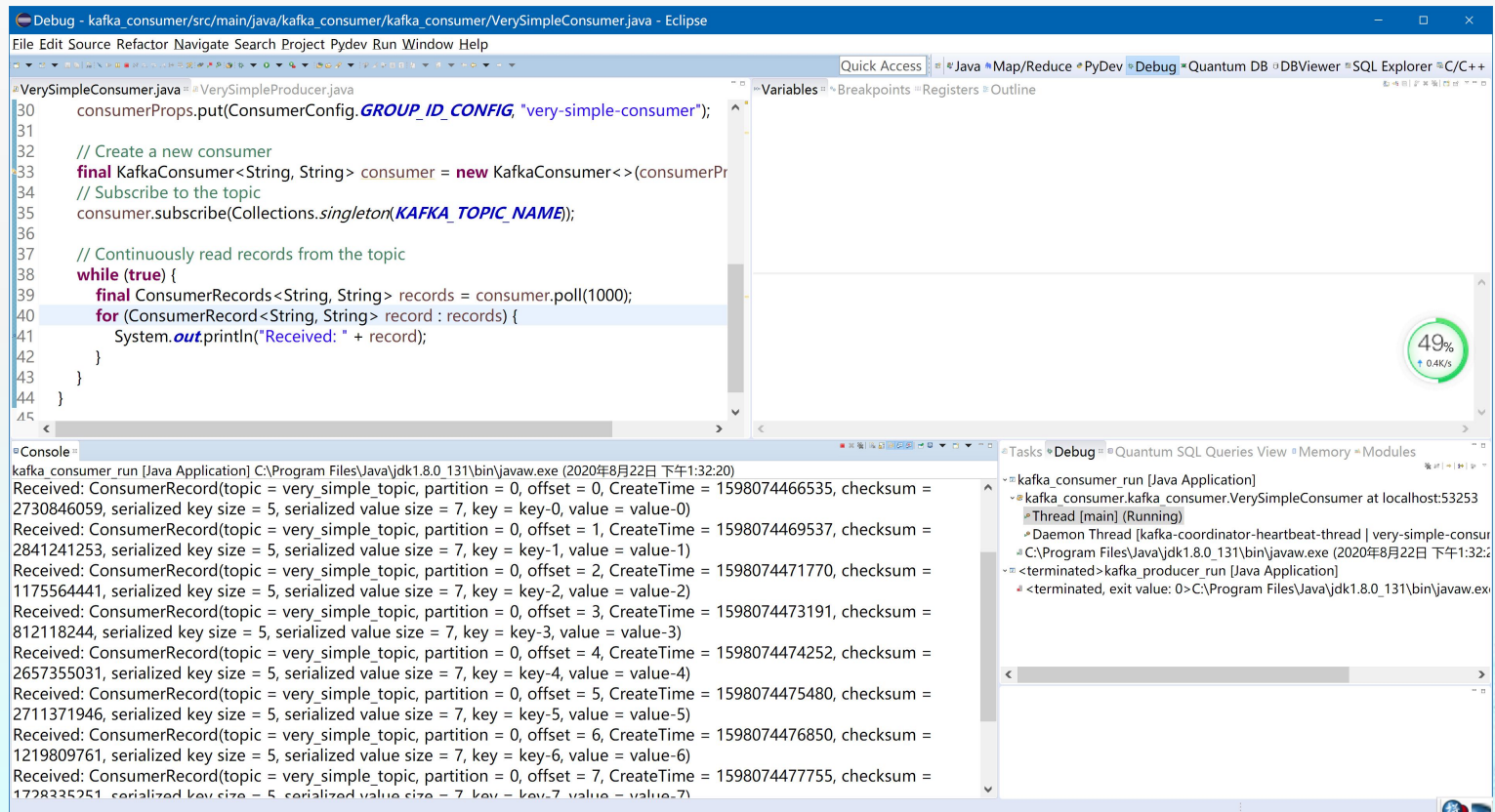
- ◆ Java开发
 - ◆ 注意，在Eclipse中，我们是可以同时调试2个应用程序的
 - ◆ 可以在Console Tab上，选择Display Selected Console，查看不同应用程序的Console输出，如图所示



Kafka

Java开发

输出结果如图



```
Debug - kafka_consumer/src/main/java/kafka_consumer/kafka_consumer/VerySimpleConsumer.java - Eclipse
File Edit Source Refactor Navigate Search Project PyDev Run Window Help

VerySimpleConsumer.java
30 consumerProps.put(ConsumerConfig.GROUP_ID_CONFIG, "very-simple-consumer");
31
32 // Create a new consumer
33 final KafkaConsumer<String, String> consumer = new KafkaConsumer<>(consumerPr
34 // Subscribe to the topic
35 consumer.subscribe(Collections.singleton(KAFKA_TOPIC_NAME));
36
37 // Continuously read records from the topic
38 while (true) {
39     final ConsumerRecords<String, String> records = consumer.poll(1000);
40     for (ConsumerRecord<String, String> record : records) {
41         System.out.println("Received: " + record);
42     }
43 }
44 }
45

Console
kafka_consumer_run [Java Application] C:\Program Files\Java\jdk1.8.0_131\bin\javaw.exe (2020年8月22日 下午1:32:20)
Received: ConsumerRecord(topic = very_simple_topic, partition = 0, offset = 0, CreateTime = 1598074466535, checksum =
2730846059, serialized key size = 5, serialized value size = 7, key = key-0, value = value-0)
Received: ConsumerRecord(topic = very_simple_topic, partition = 0, offset = 1, CreateTime = 1598074469537, checksum =
2841241253, serialized key size = 5, serialized value size = 7, key = key-1, value = value-1)
Received: ConsumerRecord(topic = very_simple_topic, partition = 0, offset = 2, CreateTime = 1598074471770, checksum =
1175564441, serialized key size = 5, serialized value size = 7, key = key-2, value = value-2)
Received: ConsumerRecord(topic = very_simple_topic, partition = 0, offset = 3, CreateTime = 1598074473191, checksum =
812118244, serialized key size = 5, serialized value size = 7, key = key-3, value = value-3)
Received: ConsumerRecord(topic = very_simple_topic, partition = 0, offset = 4, CreateTime = 1598074474252, checksum =
2657355031, serialized key size = 5, serialized value size = 7, key = key-4, value = value-4)
Received: ConsumerRecord(topic = very_simple_topic, partition = 0, offset = 5, CreateTime = 1598074475480, checksum =
2711371946, serialized key size = 5, serialized value size = 7, key = key-5, value = value-5)
Received: ConsumerRecord(topic = very_simple_topic, partition = 0, offset = 6, CreateTime = 1598074476850, checksum =
1219809761, serialized key size = 5, serialized value size = 7, key = key-6, value = value-6)
Received: ConsumerRecord(topic = very_simple_topic, partition = 0, offset = 7, CreateTime = 1598074477755, checksum =
1728335251, serialized key size = 5, serialized value size = 7, key = key-7, value = value-7)

Tasks Debug Quantum SQL Queries View Memory Modules
kafka_consumer_run [Java Application]
kafka_consumer.kafka_consumer.VerySimpleConsumer at localhost:53253
Thread [main] (Running)
Daemon Thread [kafka-coordinator-heartbeat-thread] very-simple-consum
C:\Program Files\Java\jdk1.8.0_131\bin\javaw.exe (2020年8月22日 下午1:32:20)
<terminated> kafka_producer_run [Java Application]
<terminated, exit value: 0> C:\Program Files\Java\jdk1.8.0_131\bin\javaw.exe
```

Kafka

- ◆ Java开发
 - ◆ (7) 最后停止zookeeper和Kafka broker 0

```
cd /opt/linuxsir/kafka_2.10-0.10.2.1  
bin/zookeeper-server-stop.sh config/zookeeper.properties &  
bin/kafka-server-stop.sh config/server.properties &
```

如果上述命令不能正常停止进程，可以使用如下命令找出zookeeper和kafka broker 0的进程号，然后kill掉进程。

```
netstat -ntlp|grep 9092  
kill -9 75182 \\ 75182为kafka broker 0的进程号
```

```
netstat -ntlp|grep 12182  
kill -9 74861 \\ 74861为zookeeper的进程号
```

pause



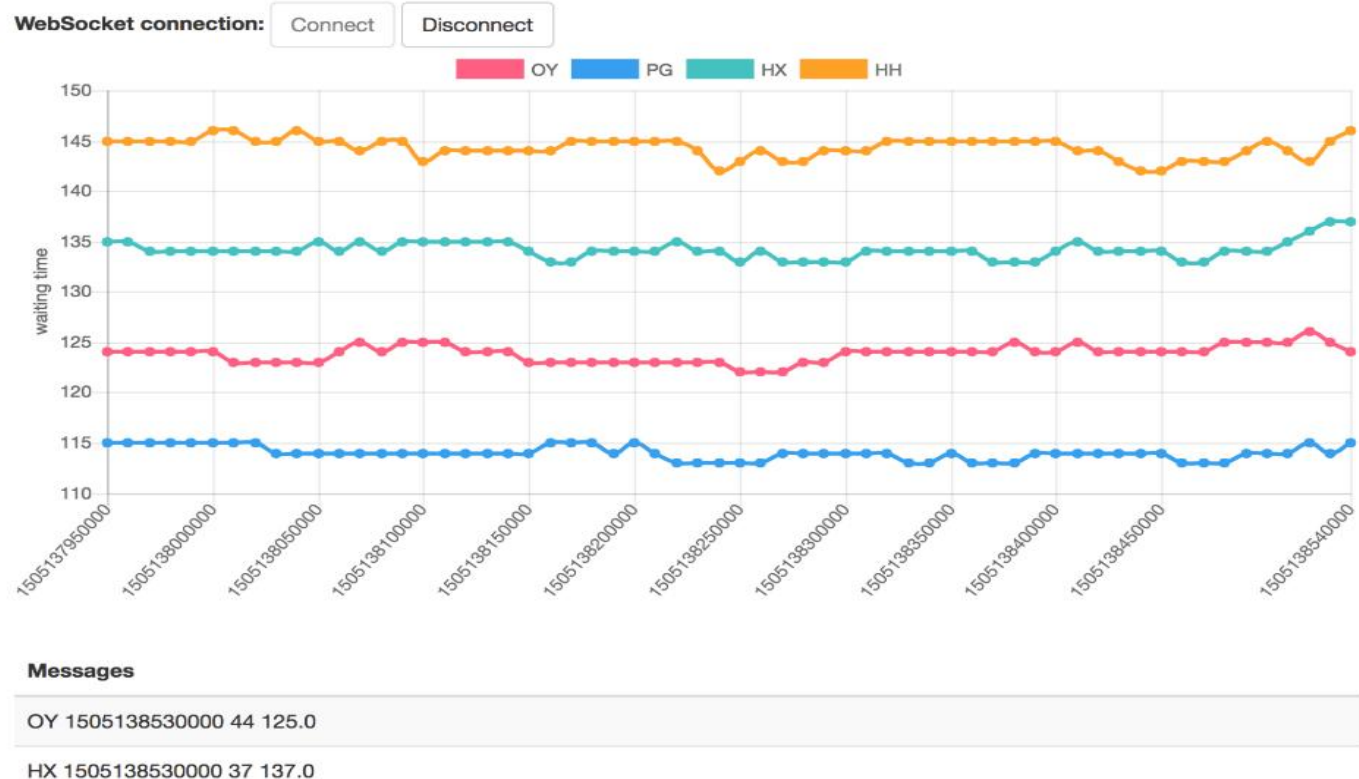
Kafka

- ◆ Java开发
 - ◆ <https://github.com/ebi-wp/kafka-streams-api-websockets>
 - ◆ 给出了一个利用Kafka Streams进行流式数据处理的实例
 - ◆ 这个实例包括几个主要的组件，(1)一个Producer，生成一系列传感器数据
 - ◆ (2)一个基于Kafka Streams API进行开发的Java应用程序，负责对数据流进行适当分组和汇总，生成临时汇总数据
 - ◆ (3)一个基于Spring Boot 进行开发的应用程序，把汇总数据读出来，以图表形式在网页上动态显示

Kafka

Java开发

localhost:8080



pause



Kafka

- ◆ Kafka与Flume的配合
 - ◆ Kafka可以作为快速到达的数据临时落地的一个暂存区
 - ◆ Flume则可以作为一个粘接剂，把Kafka里的数据，及时地导入到Hadoop的HDFS、HBase、或者Hive
- ◆ 这里介绍一个实例，如何通过Flume从Kafka把数据转移到Hive，Kafka扮演临时数据暂存的角色，Flume负责把Kafka里的数据最终转移到目的地，即Hive数据仓库
- ◆ Flume的Source是Kafka，而Flume的Sink是Hive
- ◆ 注意，Kafka的数据来源可以是各种数据源，比如从Web服务器采集的日志、从业务系统采集的订单等

Kafka

- ◆ Kafka与Flume的配合
 - ◆ (1) 准备工作
 - ◆ 为了支持Hive Sink, 需要打开/opt/linuxsir/hive/conf/hive-site.xml文件, 修改若干配置项
 - ◆ 并且把Hive的相关Jar包, 拷贝到Flume的lib目录下比如 /opt/linuxsir/apache-flume/lib
 - ◆ 请参考教材12.7节

Kafka

- ◆ Kafka与Flume的配合
 - ◆ (2) 新建Flume配置文件
- ◆ 在/opt/linuxsir/apache-flume/conf目录下，创建kafka_source_hive_sink.conf，内容如下

```
agent1.sources=kafka_source
agent1.channels=mem_channel
agent1.sinks=hive_sink

# kafka_source的配置
agent1.sources.kafka_source.type=org.apache.flume.source.kafka.KafkaSource
agent1.sources.kafka_source.zookeeperConnect=192.168.31.129:12182
agent1.sources.kafka_source.bootstrap.servers=192.168.31.129:9092
agent1.sources.kafka_source.topic=kafka_test
agent1.sources.kafka_source.channels=mem_channel
agent1.sources.kafka_source.consumer.timeout.ms=1000
agent1.sources.kafka_source.channels=mem_channel

# hive_sink的配置
agent1.sinks.hive_sink.type=hive
agent1.sinks.hive_sink.hive.metastore=thrift://192.168.31.129:19083
agent1.sinks.hive_sink.hive.database=default
agent1.sinks.hive_sink.hive.table=kafka_test
agent1.sinks.hive_sink.hive.txnsPerBatchAsk=2
agent1.sinks.hive_sink.batchSize=10
agent1.sinks.hive_sink.serializer=DELIMITED
agent1.sinks.hive_sink.serializer.delimiter=,
agent1.sinks.hive_sink.serializer.fieldnames=id,name,age
agent1.sinks.hive_sink.channel=mem_channel

# mem_channel的配置
agent1.channels.mem_channel.type=memory
agent1.channels.mem_channel.capacity=1000
agent1.channels.mem_channel.transactionCapacity=100
```

Kafka

- ◆ Kafka与Flume的配合

- ◆ (3) 建立Hive表格

- ◆ 启动Hive Shell，准备建立表格。注意，首先需要启动HDFS和YARN

```
rm -rf /opt/linuxsir/hadoop/logs/*.*  
ssh root@192.168.31.130 rm -rf /opt/linuxsir/hadoop/logs/*.*  
ssh root@192.168.31.131 rm -rf /opt/linuxsir/hadoop/logs/*.*
```

```
clear  
cd /opt/linuxsir/hadoop/sbin  
./start-dfs.sh  
./start-yarn.sh
```

```
clear  
jps  
ssh root@192.168.31.130 jps  
ssh root@192.168.31.131 jps
```

```
cd /opt/linuxsir/hive  
./bin/hive --service metastore -p 19083 &  
\\ 等待metastore启动  
netstat -lanp | grep 19083
```

```
cd $HIVE_HOME/bin  
./hive
```

Kafka

- ◆ Kafka与Flume的配合
 - ◆ (3) 建立Hive表格
 - ◆ 在Hive Shell里运行如下SQL，建立表格kafka_test，注意需要设定buckets，格式为ORC，支持transaction

```
create table kafka_test (id int, name string, age int) clustered by (id) into 2  
buckets STORED AS ORC TBLPROPERTIES ("transactional"="true");
```

```
\\退出Hive Shell  
quit;
```

Kafka

- ◆ Kafka与Flume的配合
 - ◆ (4) 启动Zookeeper和Kafka Broker 0

```
cd /opt/linuxsir/kafka_2.10-0.10.2.1  
bin/zookeeper-server-start.sh config/zookeeper.properties &
```

```
\\ 等待zookeeper启动  
netstat -ntlp|grep 12182
```

```
bin/kafka-server-start.sh config/server.properties &
```

```
\\ 等待Kafka Broker 0启动  
netstat -ntlp|grep 9092
```

```
\\ 注意Kafka Broker在192.168.31.129启动之后, 在9092端口监听
```

Kafka

- ◆ Kafka与Flume的配合
 - ◆ (5) 创建Kafka的topic

```
cd /opt/linuxsir/kafka_2.10-0.10.2.1  
bin/kafka-topics.sh --create --zookeeper 192.168.31.129:12182 --  
replication-factor 1 --partitions 1 --topic kafka_test  
bin/kafka-topics.sh --list --zookeeper 192.168.31.129:12182
```

Kafka

- ◆ Kafka与Flume的配合
 - ◆ (6) 打开一个终端窗口，通过如下命令
 - ◆ 启动Flume Server，并启动agent1

```
cd /opt/linuxsir  
cd apache-flume  
bin/flume-ng agent --conf ./conf/ -f ./conf/kafka_source_hive_sink.conf \  
-Dflume.root.logger=DEBUG,console -n agent1
```

\\ 注意按Ctrl+C可以停止Flume Server

Kafka

- ◆ Kafka与Flume的配合
 - ◆ (7) 启动第二个终端，运行Kafka Producer。

```
cd /opt/linuxsir
cd kafka_2.10-0.10.2.1
bin/kafka-console-producer.sh --broker-list 192.168.31.129:9092 --topic
kafka_test
\\ 输入的格式如下
1,john,25
2,mary,26
3,june,27
4,april,28
\\ 输入以后回车继续输入

\\ 注意按Ctrl+C可以停止Kafka Producer
```

Kafka

- ◆ Kafka与Flume的配合
 - ◆ (8) 打开第三个终端，启动Hive Shell
 - ◆ 执行如下查询，看看数据是否已经插入到表格中

```
cd $HIVE_HOME/bin  
./hive  
  
select * from kafka_test;  
exit;
```

Kafka

- ◆ Kafka与Flume的配合
 - ◆ (8) 打开第三个终端，启动Hive Shell
 - ◆ 执行如下查询，看看数据是否已经插入到表格中
 - ◆ 查询到的结果如下

OK

2 mary 26

4 april 28

1 john 25

3 june 27

Time taken: 0.459 seconds, Fetched: 7 row(s)

Kafka

- ◆ Kafka与Flume的配合
 - ◆ (9) 停止Kafka Broker 0和Zookeeper

```
cd /opt/linuxsir/kafka_2.10-0.10.2.1  
bin/zookeeper-server-stop.sh config/zookeeper.properties &  
bin/kafka-server-stop.sh config/server.properties &
```

Kafka

- ◆ Kafka与Flume的配合
 - ◆ (10) 停止metastore、YARN、HDFS。

```
netstat -lanp | grep 19083  
kill -9 16336 \\16336是metastore进程号
```

```
cd /opt/linuxsir/hadoop/sbin  
./stop-yarn.sh  
./stop-dfs.sh
```

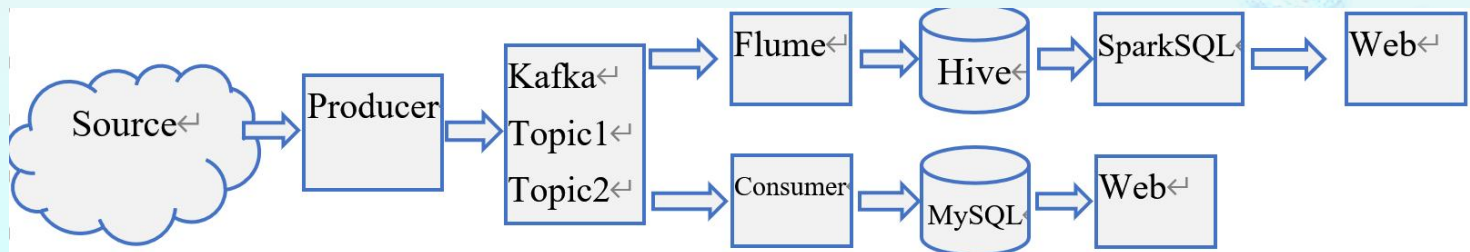
```
clear  
jps  
ssh root@192.168.31.130 jps  
ssh root@192.168.31.131 jps
```

pause



Kafka

- ◆ 流式处理与批处理的结合
- ◆ 有了Kafka和Flume，我们就可以把流式处理和批处理结合起来
- ◆ (1)开发Kafka Producer程序，从数据源拉取数据，分别发送到Kafka Topic1和Topic2
- ◆ (2)利用Flume把数据从Topic1转移到Hive
- ◆ (3)在网页上提交SQL查询，利用SparkSQL对Hive数据进行查询分析，以图表形式展现查询结果
- ◆ (4)开发Kafka Consumer程序，从Topic2拉取数据，自行进行分组统计，写入MySQL
- ◆ (5)前端网页可以定时查询MySQL，以图表的形式显示汇总结果
- ◆ (4)和(5)也可以采用前文介绍的方式，即利用Kafka Stream API进行流式处理，然后使用Spring Boot实时地把结果显示在网页上



利用 Kafka Streams API 对数据流进行处理